

第三章 MapReduce习题解答



毕倪飞、杨溢

華東師範大學



大纲

2

- 计算课程平均分
- 最大温度
- K均值聚类
- 网页链接排名
- 消费者统计



计算课程平均分

3

- 要求：某班级有若干名同学，共修读三门必修课，现需计算这三门课程的班级平均成绩（向下取整）
- 输入：学号,数学分析分数,概率论分数,实变函数分数
- 输出：课程名,平均分

输入	输出
10160001,98,80,75 10160002,53,94,77 10160003,61,86,91	Function of Real Variable 81 Mathematical analysis 70 Probability Theory 86

解决方案

4

□ Map过程:

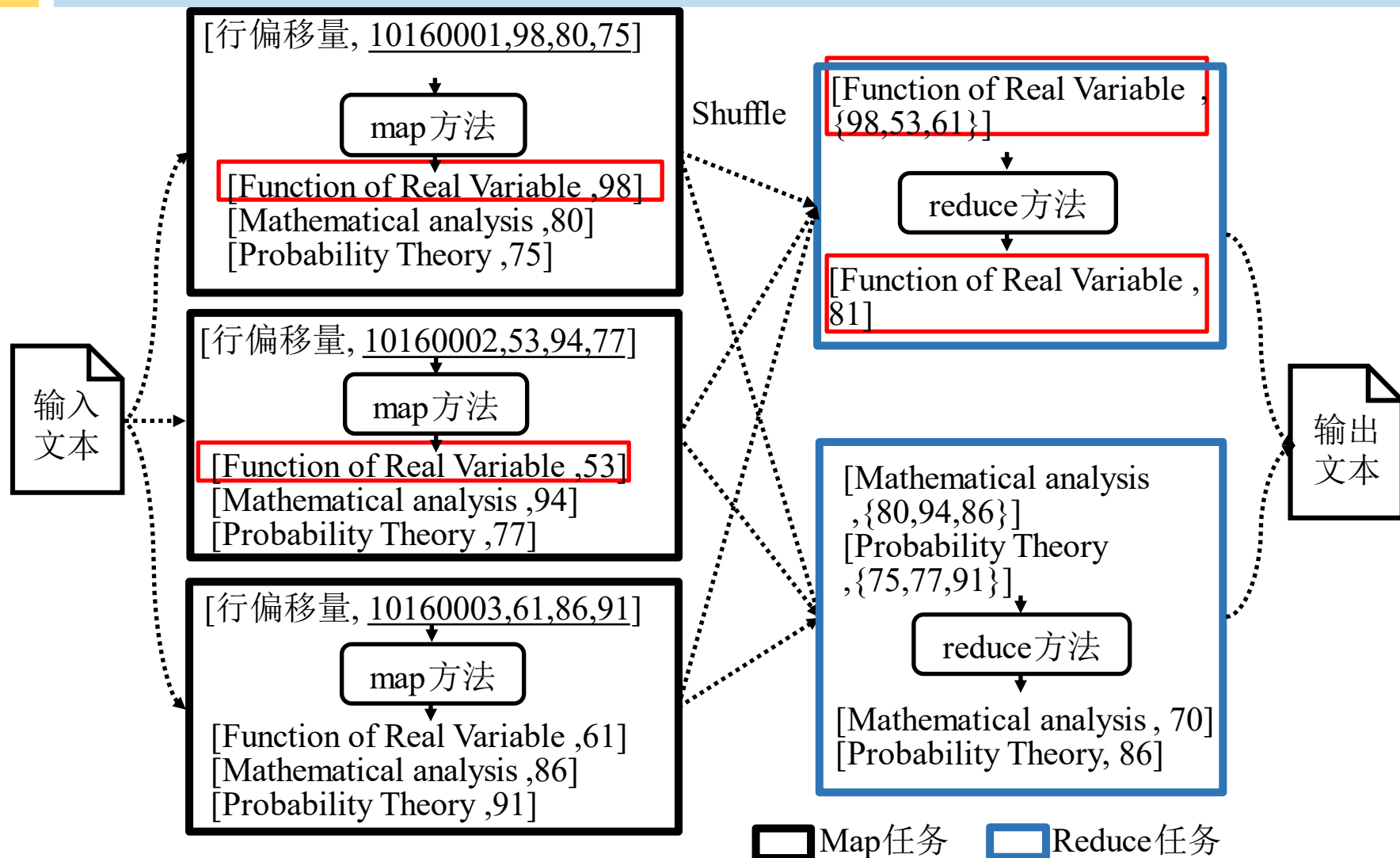
- ✚ 把每个同学的成绩转换为[课程名, 分数]键值对

□ Reduce过程:

- ✚ 相同课程的键值对被发送到同一个Reduce任务
- ✚ 计算课程总分并求得课程平均分
- ✚ 输出最终的结果[课程名, 平均分]键值对

计算课程平均分的运行过程

5



编写map方法

6

```
1 public class ScoreMapperImpl extends ScoreMapper {
2
3     @Override
4     public void map(Object key, Text value, Context context)
5         throws IOException, InterruptedException {
6         String[] strs = value.toString().split(",");
7         for (int i = 1; i < strs.length; i++) {
8             Text course = new Text(getCourseName(i - 1));
9             IntWritable score = new IntWritable(Integer.parseInt(strs[i]));
10            context.write(course, score);
11        }
12    }
13
14 }
```

编写reduce方法

7

```
1 public class ScoreReducerImpl extends ScoreReducer {
2
3     @Override
4     public void reduce(Text key, Iterable<IntWritable> values, Context context)
5         throws IOException, InterruptedException {
6         int sum = 0;
7         int count = 0;
8         for (IntWritable val : values) {
9             sum += val.get();
10            count++;
11        }
12        context.write(key, new IntWritable(sum / count));
13    }
14
15 }
```

编写run方法

8

```
1
2 public class ScoreRunner extends Configured implements Tool {
3
4     @Override
5     public int run(String[] args) throws Exception {
6         Job job = Job.getInstance(getConf(), getClass().getSimpleName());
7         job.setJarByClass(getClass());
8
9         FileInputFormat.addInputPath(job, new Path(args[0]));
10        FileOutputFormat.setOutputPath(job, new Path(args[1]));
11
12        job.setMapperClass(ScoreMapperImpl.class);
13        job.setReducerClass(ScoreReducerImpl.class);
14
15        job.setMapOutputKeyClass(Text.class);
16        job.setMapOutputValueClass(IntWritable.class);
17
18        job.setOutputKeyClass(Text.class);
19        job.setOutputValueClass(IntWritable.class);
20
21        return job.waitForCompletion(true) ? 0 : 1;
22    }
23
24 }
25
```



大纲

9

- 计算课程平均分
- 最大温度
- K均值聚类
- 网页链接排名
- 消费者统计



最大温度

10

- 要求：给定一份年份与温度的统计表，现需计算统计表中每个年份的最大温度。
- 输入：每行包含一个年份和温度
- 输出：每行包含一个年份及其最高温度

输入	输出
1990 21	1990 25
1990 18	1991 35
1991 35	1992 40
1992 30	
1990 25	
1992 21	
1992 40	

解决方案

11

□ Map过程:

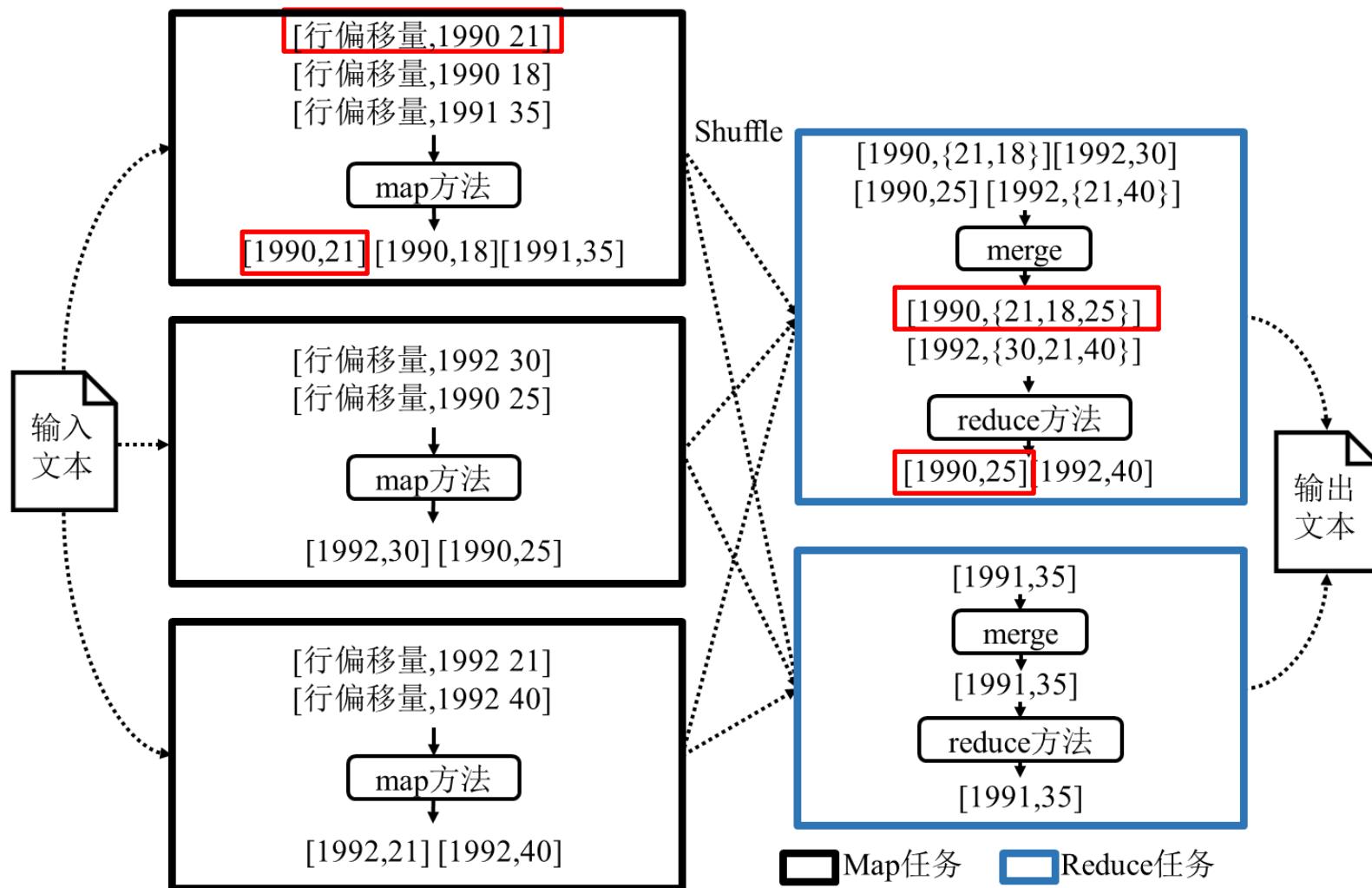
- ✚ 把文本的每行内容转换为[年份,温度]键值对

□ Reduce过程:

- ✚ 年份相同的键值对被发送到同一个Reduce任务
- ✚ 遍历年份相同的键值对，通过比较来获取每个年份所对应的最高温度
- ✚ 输出最终的结果[年份, 最高温度]键值对

最大温度的运行过程

12



编写map方法

13

```
1 public class MaxTemperatureMapperImpl extends MaxTemperatureMapper {
2
3     @Override
4     public void map(Object key, Text value, Context context)
5         throws IOException, InterruptedException {
6         String line = value.toString();
7         if (!"" equals(line)) {
8             String year = line.substring(0, 4);
9             int temperature = Integer.parseInt(line.substring(5));
10            context.write(new Text(year), new IntWritable(temperature));
11        }
12    }
13 }
```

编写reduce方法

14

```
1 public class MaxTemperatureReducerImpl extends MaxTemperatureReducer {
2
3     @Override
4     public void reduce(Text key, Iterable<IntWritable> values, Context context)
5         throws IOException, InterruptedException {
6         int maxValue = Integer.MIN_VALUE;
7         for (IntWritable value : values) {
8             maxValue = Math.max(maxValue, value.get());
9         }
10        context.write(key, new IntWritable(maxValue));
11    }
12 }
13
```

编写run方法

15

```
1 public class MaxTemperatureRunner extends Configured implements Tool {
2
3     @Override
4     public int run(String[] args) throws Exception {
5         Job job = Job.getInstance(getConf(), getClass().getSimpleName());
6         job.setJarByClass(getClass());
7
8         FileInputFormat.addInputPath(job, new Path(args[0]));
9         FileOutputFormat.setOutputPath(job, new Path(args[1]));
10
11         job.setMapperClass(MaxTemperatureMapperImpl.class);
12         job.setReducerClass(MaxTemperatureReducerImpl.class);
13
14         job.setMapOutputKeyClass(Text.class);
15         job.setMapOutputValueClass(IntWritable.class);
16
17         job.setOutputKeyClass(Text.class);
18         job.setOutputValueClass(IntWritable.class);
19
20         return job.waitForCompletion(true) ? 0 : 1;
21     }
22 }
```



大纲

16

- 计算课程平均分
- 最大温度
- K均值聚类
- 网页链接排名
- 消费者统计



K均值聚类

17

- 要求：相邻两次迭代的聚类中心之间的欧式距离小于某一阈值(0.05)后终止迭代
- 输入：一个数据集和一个聚类中心集
 - ✚ 数据集：每行为一个n维数据点及其类别标签
 - ✚ 聚类中心集：每行为一个n维数据点
- 输出：n维数据点及其所属类别

数据集	聚类中心集	聚类结果
0,0 -1	1,2 3,1	0,0 0
1,2 -1		1,2 0
3,1 -1		3,1 0
8,8 -1		8,8 1
9,10 -1		9,10 1
10,7 -1		10,7 1

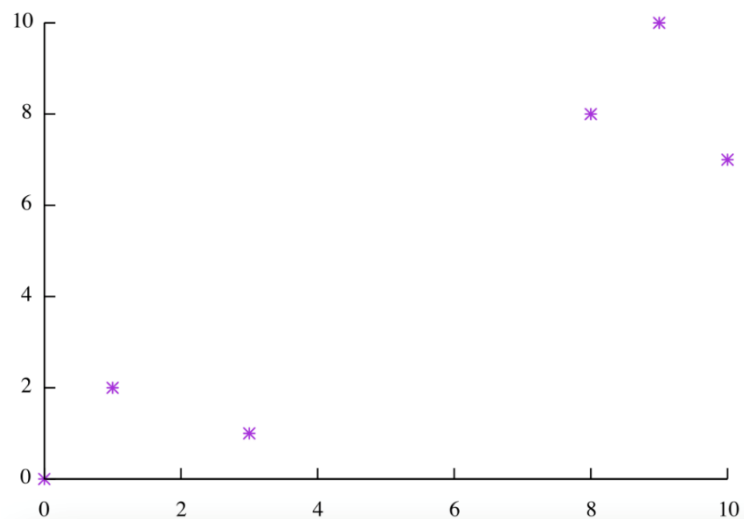


K均值聚类算法回顾

18

□ 算法执行过程

1. 设定聚类中心数 k 。例如, $k=2$

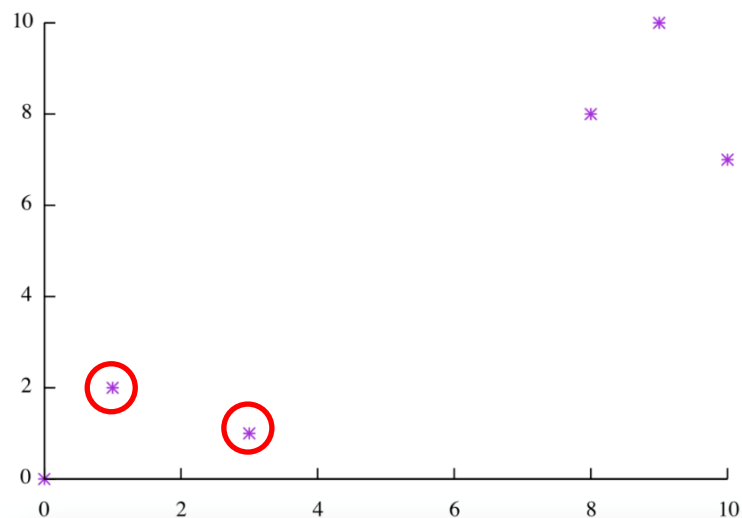


K均值聚类算法回顾

19

□ 算法执行过程

1. 设定聚类中心数 k 。例如, $k=2$
2. 选取聚类中心

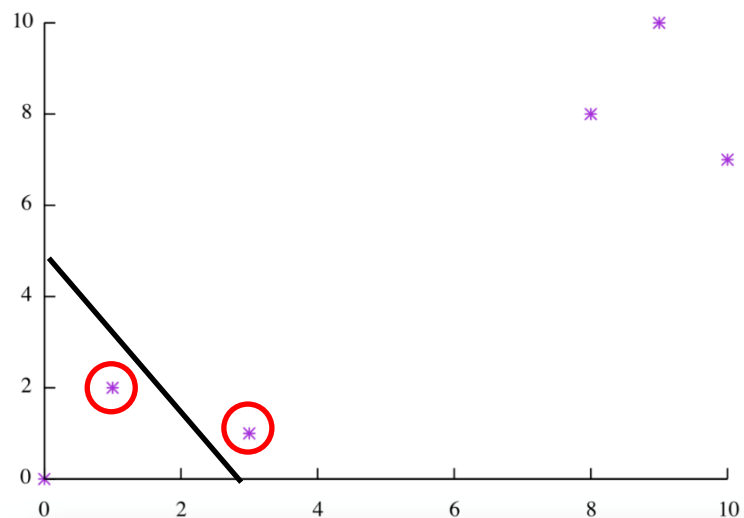


K均值聚类算法回顾

20

□ 算法执行过程

1. 设定聚类中心数 k 。例如, $k=2$
2. 选取聚类中心
3. 寻找每个数据点距离最近的聚类中心

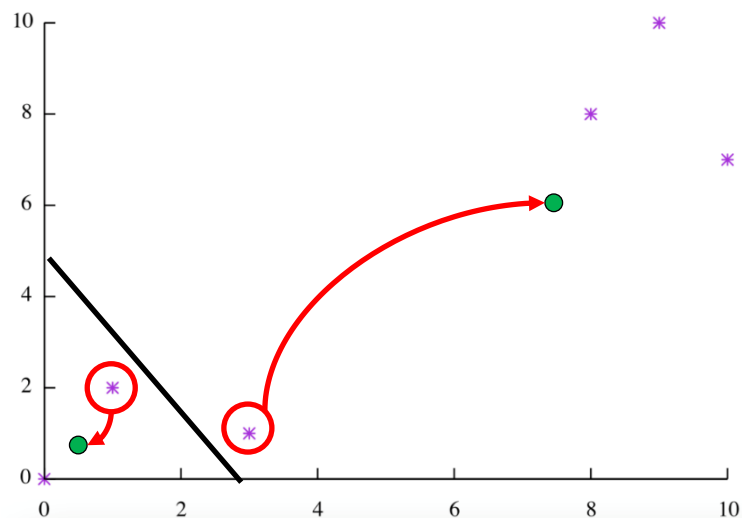


K均值聚类算法回顾

21

□ 算法执行过程

1. 设定聚类中心数 k 。例如, $k=2$
2. 选取聚类中心
3. 寻找每个数据点距离最近的聚类中心
4. 计算同属一个聚类中心的数据点的新的聚类中心

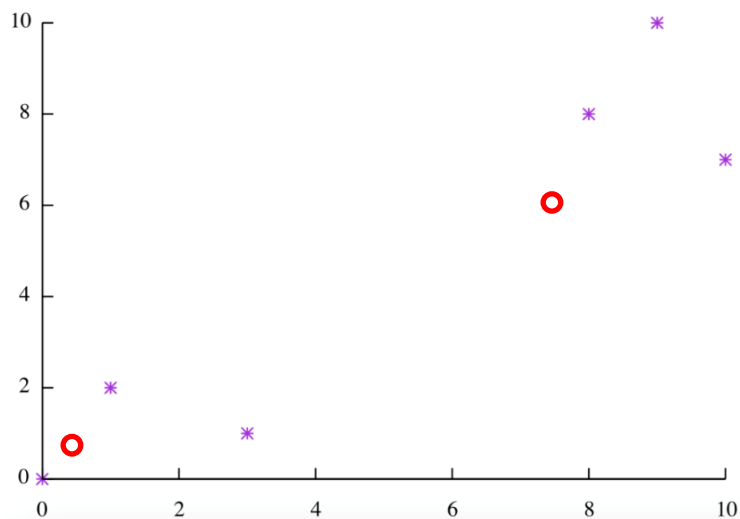


K均值聚类算法回顾

22

□ 算法执行过程

1. 设定聚类中心数 k 。例如, $k=2$
2. 选取聚类中心
3. 寻找每个数据点距离最近的聚类中心
4. 计算同属一个聚类中心的数据点的新的聚类中心
5. 重复3-4直到相邻两次迭代的聚类中心之间的欧式距离小于某一阈值



解决方案

23

□ 算法执行过程

1. 设定聚类中心数 k 。例如, $k=2$

2. 选取聚类中心

一次迭代一个MapReduce作业

3. 寻找每个数据点距离最近的聚类中心

在Map阶段实现, 其中采用广播方式读取聚类中心

4. 计算同属一个聚类中心的数据点的新的聚类中心

在Reduce阶段实现

5. 重复3-4直到相邻两次迭代的聚类中心之间的欧式距离小于某一阈值

在KmeansRunner中的循环语句中进行控制



解决方案

24

□ Map过程:

- 广播方式读取聚类中心集，并计算每个数据点与各聚类中心之间的距离，为数据点添加类别标签将数据点转换为 [类别号, 数据点] 的键值对
- 若相邻两次迭代的聚类中心之间的欧式距离小于某一阈值，则输出聚类结果；否则，输出迭代中间结果

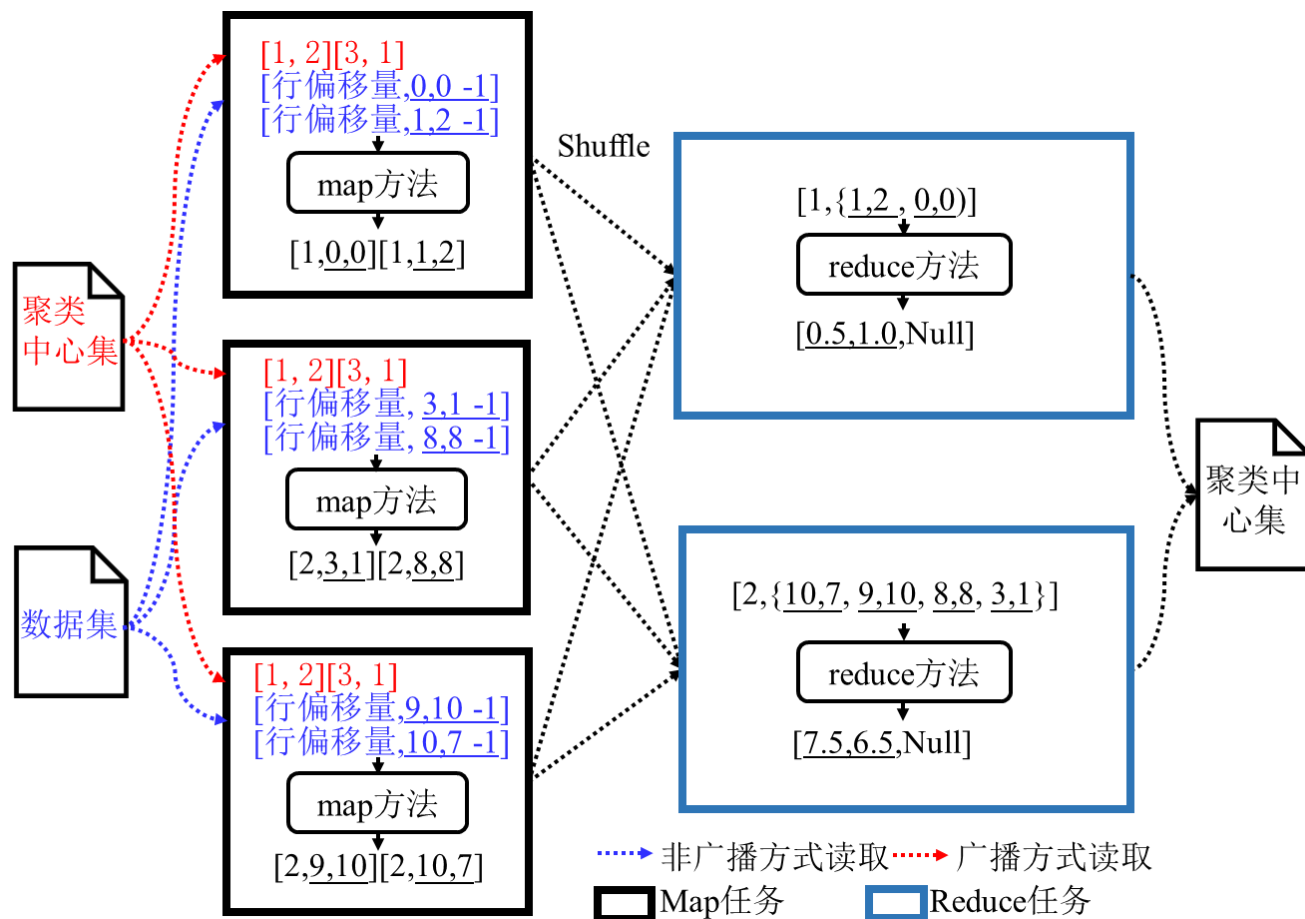
□ Reduce过程:

- 为属于同一聚类中心的数据点计算新的聚类中心，并输出[新聚类中心, 空值]的键值对

单次迭代运行过程

25

“广播”聚类中心集



编写map方法

26

```
1 public class KMeansMapperImpl extends KMeansMapper {
2     private List<List<Double>> centers = new ArrayList<>();
3
4     @Override
5     public void map(LongWritable key, Text value, Context context)
6         throws IOException, InterruptedException {
7         String[] dimensions;
8         List<Double> point = new ArrayList<>();
9         int centerIndex = 1;
10        double minDistance = Double.MAX_VALUE;
11
12        if (centers.size() == 0) {
13            // 获取广播的聚类中心集路径
14            String centersPath = context.getCacheFiles()[0].toString();
15            // 将聚类中心加载到集合centers
16            centers = CentroidsOperation.getCenters(centersPath, true);
17        }
18
19        // 解析数据点
20        dimensions = value.toString().split("[,\\t]");
21        for (int i = 0; i < dimensions.length - 1; i++) {
22            point.add(Double.parseDouble(dimensions[i]));
23        }
24
25        // 遍历聚类中心集并计算与数据点的距离
26        for (int i = 0; i < centers.size(); i++) {
27            double distance = 0;
28            List<Double> center = centers.get(i);
29            // 计算数据点与当前聚类中心之间的距离
30            for (int j = 0; j < center.size(); j++) {
31                distance += Math.pow((point.get(j) - center.get(j)), 2);
32            }
33            distance = Math.sqrt(distance);
34            // 如果距离小于当前记录的最小距离则将数据点分配给当前聚类中心（类别号标识）
35            if (distance < minDistance) {
36                minDistance = distance;
37                centerIndex = i;
38            }
39        }
40
41        // 从输入值中截取数据点
42        String pointData = value.toString().split("\\t")[0];
43        if (KMeansRunner.compareResult) {
44            context.write(new Text(pointData), new Text(String.valueOf(centerIndex)));
45        } else {
46            // 输出以类别号为键，数据点为值的键值对
47            context.write(new Text(String.valueOf(centerIndex)), new Text(pointData));
48        }
49    }
50 }
```



编写reduce方法

27

```
1 public class KMeansReducerImpl extends KMeansReducer {
2     @Override
3     public void reduce(Text key, Iterable<Text> values, Context context)
4         throws IOException, InterruptedException {
5         List<List<Double>> points = new ArrayList<>();
6         // 解析数据点并保存到集合points
7         for (Text text : values) {
8             String value = text.toString();
9             List<Double> point = new ArrayList<>();
10            for (String s : value.split(",")) {
11                point.add(Double.parseDouble(s));
12            }
13            points.add(point);
14        }
15
16        StringBuilder newCenter = new StringBuilder();
17
18        // 计算每个维度的平均值从而得到新的聚类中心
19        for (int i = 0; i < points.get(0).size(); i++) {
20            double sum = 0;
21            // 计算第i个维度值的和
22            for (List<Double> data : points) {
23                sum += data.get(i);
24            }
25            // 计算平均值得到新的聚类中心的第i个维度值并生成需要输出的数据
26            newCenter.append(sum / points.size());
27            newCenter.append(",");
28        }
29        context.write(new Text(newCenter.toString()), NullWritable.get());
30    }
31 }
```



编写run方法

28

```
1  public int mainRun(String[] args) throws Exception {
2      int exitCode = 0;
3      sigma = Double.parseDouble(args[3]);
4      // 执行指定次数的迭代, 在最后一次迭代时输出聚类结果
5      while (!compareResult) {
6          exitCode = ToolRunner.run(new KMeansRunner(), args);
7          if (exitCode == -1) {
8              break;
9          }
10         //判断迭代前后聚类中心之间的距离是否小于阈值
11         if (iteration == 0) {
12             compareResult = CentersOperation.compareCenters(args[2], args[1] + iteration);
13         } else {
14             compareResult = CentersOperation.compareCenters(args[1] + (iteration - 1), args[1] +
15             iteration);
16         }
17         iteration++;
18     }
19     ToolRunner.run(new KMeansRunner(), args);
20     return exitCode;
21 }
```



大纲

29

- 计算课程平均分
- 最大温度
- K均值聚类
- 网页链接排名
- 消费者统计



网页链接排名

30

- 输入：两个输入文本分别保存网页链接关系和网页排名。
- 输出：网页名及其排名值

输入		输出
A B D	A 1	A 0.21436
B C	B 1	B 0.36332
C A B	C 1	C 0.40833
D B C	D 1	D 0.13027

输入	输出
A 1.0 B 1.0 D 1.0	A 0.21436
B 1.0 C 1.0	B 0.36332
C 1.0 A 1.0 B 1.0	C 0.40833
D 1.0 B 1.0 C 1.0	D 0.13027

课本上的关系链接
排名的输入和输出

解决方案

31

□ 算法执行过程

✚ 初始时，对网页链接关系和网页排名执行连接操作

✚ 每一步迭代，网页 p_j 对其链向的网页的排名值贡献 $PR(p_j)/L(p_j)$

✚ 每个网页 p_i 累加所有链向 p_i 的网页 $M(p_i)$ 的贡献值，然后更新排名值为 $0.15/N + 0.85 * \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$

一次迭代一个MapReduce作业

✚ 最终收敛

解决方案

32

□ Map过程:

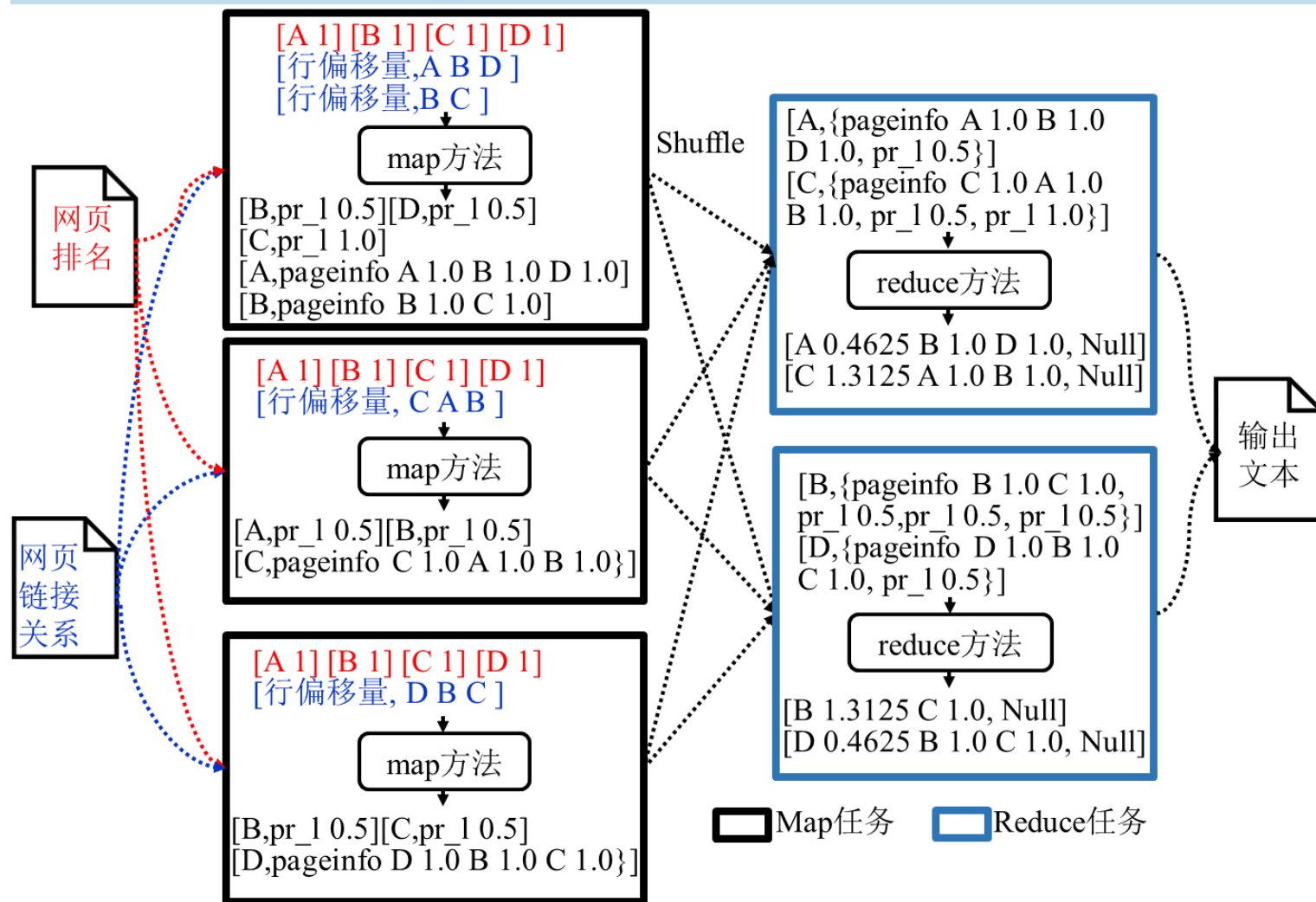
- ✚ 利用广播机制获取网页排名，初始时对网页链接关系和排名执行连接操作以形成网页信息
- ✚ 把每项网页信息转换为键值对[网页名, {PAGE_INFO, 网页信息}]
- ✚ 计算每个网页对其出站链接的贡献值

□ Reduce过程:

- ✚ 计算每个网页的排名值
- ✚ 如果不是最后一次迭代，则输出更新排名值后的网页信息。否则，输出网页名及其排名值

第一次迭代运行过程

33



编写map方法

34

```
1 public class PageRankMapperImpl extends PageRankMapper {
2     @Override
3     public void map(LongWritable key, Text value, Context context)
4         throws IOException, InterruptedException {
5
6         List<Rank> ranks = new ArrayList<>();
7
8         String[] pageInfo;
9         if (PageRankRunner.iteration == 0) {
10             String[] data = value.toString().split(" ");
11             // 获取广播的聚类中心集路径
12             String pagerank = context.getCacheFiles()[0].toString();
13             // 将聚类中心加载到集合centers
14             ranks = RanksOperation.getRanks(pagerank, false);
15             String rankValue = "";
16             String pageName = data[0];
17
18             //连接网页名与网页排名
19             for (int i = 0; i < ranks.size(); i++) {
20                 if (ranks.get(i).getPageName().equals(pageName)) {
21                     rankValue = String.valueOf(ranks.get(i).getRank());
22                     break;
23                 }
24             }
25             pageInfo = new String[data.length * 2];
26             for (int i = 0; i < data.length; i++) {
27                 pageInfo[i * 2] = data[i];
28                 if (i == 0) {
29                     pageInfo[i * 2 + 1] = rankValue;
30                 } else {
31                     pageInfo[i * 2 + 1] = "1.0";
32                 }
33             }
34         } else {
35             pageInfo = value.toString().split(" ");
36         }
37     }
```

与课本上的网页链接排名相比，map方法中增加了连接操作

```
38 // 网页的出站链接数
39 int outLink = (pageInfo.length - 2) / 2;
40 double pageRank = Double.parseDouble(pageInfo[1]);
41 ReducePageRankWritable writable;
42 writable = new ReducePageRankWritable();
43 // 计算贡献值并保存
44 writable.setData(String.valueOf(pageRank / outLink));
45 // 设置对应标识
46 writable.setTag(ReducePageRankWritable.PR_L);
47 // 对于每一个出站链接，输出贡献值
48 for (int i = 2; i < pageInfo.length; i += 2) {
49     context.write(new Text(pageInfo[i]), writable);
50 }
51 writable = new ReducePageRankWritable();
52 // 保存网页信息并标识
53 StringBuilder pageInfoResult = new StringBuilder();
54 if (PageRankRunner.iteration == 0) {
55     for (String data : pageInfo) {
56         pageInfoResult.append(data).append(" ");
57     }
58     writable.setData(pageInfoResult.toString());
59     writable.setData(value.toString());
60 }
61 }
62
63 writable.setTag(ReducePageRankWritable.PAGE_INFO);
64 // 以输入的网页信息的网页名称为key进行输出
65 context.write(new Text(pageInfo[0]), writable);
66 }
67 }
```



编写reduce方法

35

```
1 public class PageRankReducerImpl extends PageRankReducer {
2     // 阻尼系数
3     private static final double D = 0.85;
4
5     @Override
6     public void reduce(Text key, Iterable<ReducePageRankWritable> values, Context context)
7         throws IOException, InterruptedException {
8         /* 步骤2: 编写处理逻辑将[K2,V2]转换为[K3,V3]并输出 */
9         String[] pageInfo = null;
10        // 从配置项中读取网页的总数
11        int totalPage = context.getConfiguration().getInt(PageRankRunner.TOTAL_PAGE, 0);
12        // 从配置项中读取当前迭代步数
13        int iteration = context.getConfiguration().getInt(PageRankRunner.ITERATION, 0);
14        double sum = 0;
15        for (ReducePageRankWritable value : values) {
16            String tag = value.getTag();
17            // 如果是贡献值则进行求和, 否则以空格为分隔符切分后保存到pageInfo
18            if (tag.equals(ReducePageRankWritable.PR_L)) {
19                sum += Double.parseDouble(value.getData());
20            } else if (tag.equals(ReducePageRankWritable.PAGE_INFO)) {
21                pageInfo = value.getData().split(" ");
22            }
23        }
24        // 根据公式计算排名值
25        double pageRank = (1 - D) / totalPage + D * sum;
26        // 更新网页信息中的排名值
27        pageInfo[1] = String.valueOf(pageRank);
28        // 最后一次迭代输出网页名及排名值, 而其余迭代输出网页信息
29        StringBuilder result = new StringBuilder();
30        if (iteration == (PageRankRunner.MAX_ITERATION - 1)) {
31            result.append(pageInfo[0]).append(" ").append(String.format("%.5f", pageRank));
32        } else {
33            for (String data : pageInfo) {
34                result.append(data).append(" ");
35            }
36        }
37        context.write(new Text(result.toString()), NullWritable.get());
38    }
39 }
```

与课本上的网页链接排名的reduce方法完全一样



大纲

36

- 计算课程平均分
- 最大温度
- K均值聚类
- 网页链接排名
- 消费者统计



消费者统计

37

- 输入：某超市有一份消费数据，该数据包括消费者id、消费时间、消费金额以及消费者的会员属性字段。每条数据表示某一顾客的一笔交易信息。其中，假设消费金额均为整数，最小为1；超市交易量很大，超过30亿。
- 输出：会员和非会员的人数以及在超市内的消费金额总数

输入	输出
3819 2021-03-27 21:30 357 vip	non-vip 1 77 vip 1 714
3819 2021-03-27 21:32 357 vip	
3231 2021-03-27 21:30 77 non-vip	

解决方案

38

□ Map过程:

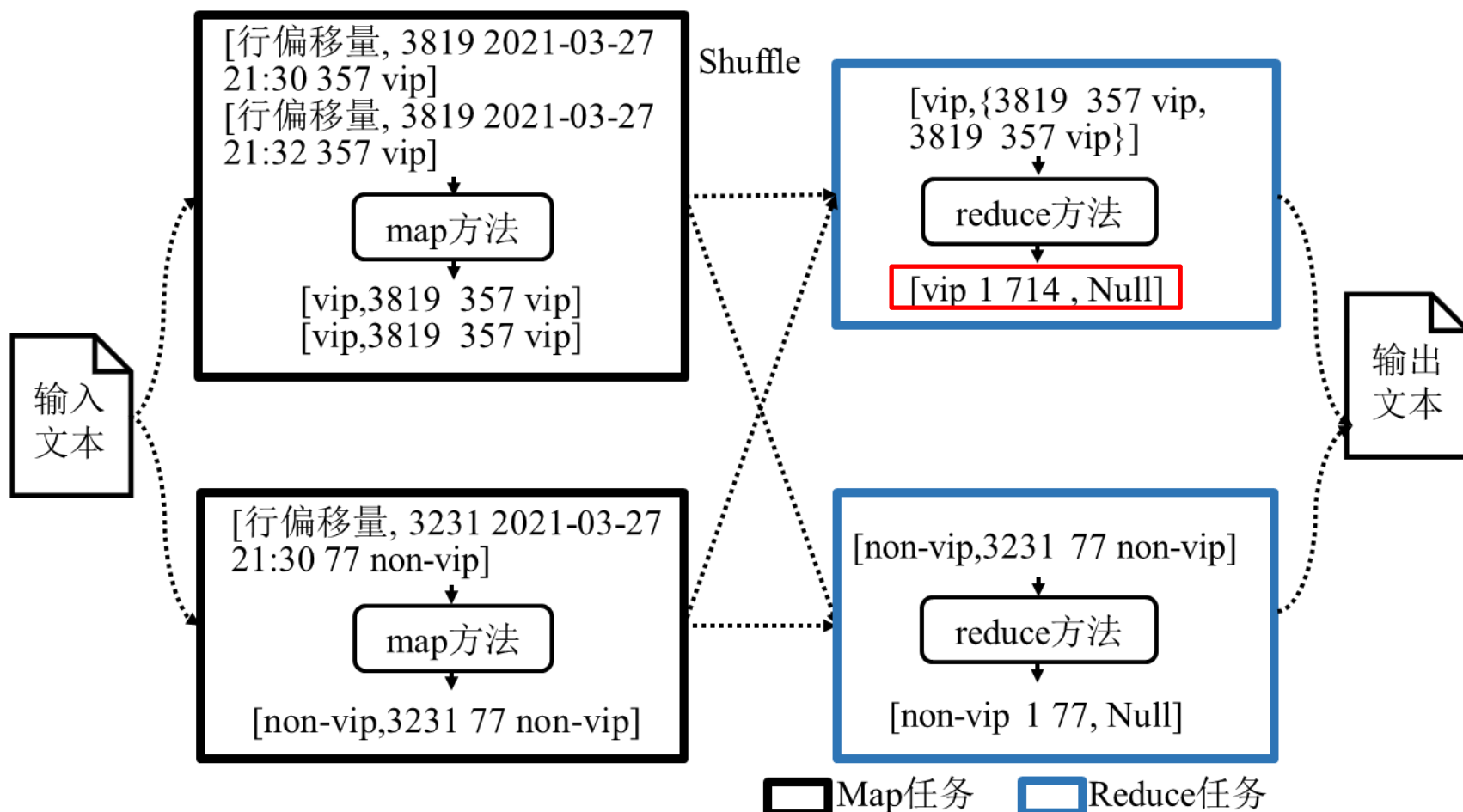
- ✚ 将每条消费数据转换为键值对[会员/非会员, 消费者信息]

□ Reduce过程:

- ✚ 根据消费者id去重并统计会员/非会员人数
- ✚ 消费金额求和
- ✚ 输出人数和消费金额总数

消费者统计的运行过程

39



编写map方法

40

```
1 public class ConsumerMapperImpl extends ConsumerMapper {
2
3     @Override
4     public void map(LongWritable key, Text value, Context context)
5         throws IOException, InterruptedException {
6         // 解析输入数据, 区分vip用户和非vip用户
7         String line = value.toString();
8         String[] fields = line.split("\t");
9         Consumer consumer = new Consumer();
10        consumer.setId(fields[0]);
11        consumer.setMoney(Integer.parseInt(fields[2]));
12        consumer.setVip(VIP.equals(fields[3]));
13        if (consumer.isVip()) {
14            context.write(new Text(VIP), consumer);
15        } else {
16            context.write(new Text(NON_VIP), consumer);
17        }
18    }
19 }
```

对消费数据进行分类以汇聚会员的消费数据和非会员的消费数据

编写reduce方法

41

对消费者进行去重并计算消费总额

```
1 public class ConsumerReducerImpl extends ConsumerReducer {
2
3     @Override
4     protected void reduce(Text key, Iterable<Consumer> values, Context context)
5         throws IOException, InterruptedException {
6         // 消费数据中存在重复的用户，因此用Set集合去重，然后计算出消费总额
7         Set<String> consumerIds = new HashSet<>();
8         long amounts = 0;
9         for (Consumer consumer : values) {
10             consumerIds.add(consumer.getId());
11             amounts += consumer.getMoney();
12         }
13         context.write(new Text(
14             key.toString() + "\\t" + consumerIds.size() + "\\t" + amounts), NullWritable.get());
15     }
16 }
17
```

谢谢! Q&A

